

Task 3: Static Timing Analysis (STA) & Timing Closure

Objective

The objective of this task is to perform Static Timing Analysis (STA) on a synthesized digital design using an open-source STA tool. The analysis focuses on timing constraints, clock definition, setup and hold timing concepts, and identifying potential timing issues. Timing closure techniques are also studied and documented.

Tools Used

- Ubuntu 24.04 LTS
- Yosys 0.33 (Synthesis Tool)
- OpenSTA 3.1.0 (Static Timing Analysis Tool)
- GVim / Nano Editor

Design Under Test

An 8-bit ALU design was used for timing analysis. The RTL design was first synthesized using Yosys to generate a gate-level netlist. A clocked version of the ALU was created to enable timing analysis.

Timing Constraints (SDC)

A timing constraint file was created to define the design clock.

Example SDC Constraint:

```
create_clock -name clk -period 10 [get_ports clk]
```

The clock period was defined as 10 ns. This constraint allows STA tools to evaluate timing paths against the specified clock frequency.

Synthesis Process

The RTL design was synthesized using Yosys. The following synthesis script was used:

```
read_verilog alu_sta.v
synth -top alu_sta
clean
write_verilog -noattr alu_sta_netlist.v
```

The synthesis process successfully generated a gate-level netlist.

Static Timing Analysis Procedure

1. Generated the synthesized netlist using Yosys.
2. Created timing constraints using an SDC file.
3. Prepared an OpenSTA TCL script.
4. Loaded the netlist into OpenSTA.
5. Linked the design and applied timing constraints.
6. Executed timing analysis commands.

OpenSTA Script:

```
read_verilog alu_sta_netlist.v
link_design alu_sta
read_sdc constraints_sta.sdc
report_checks
```

Results

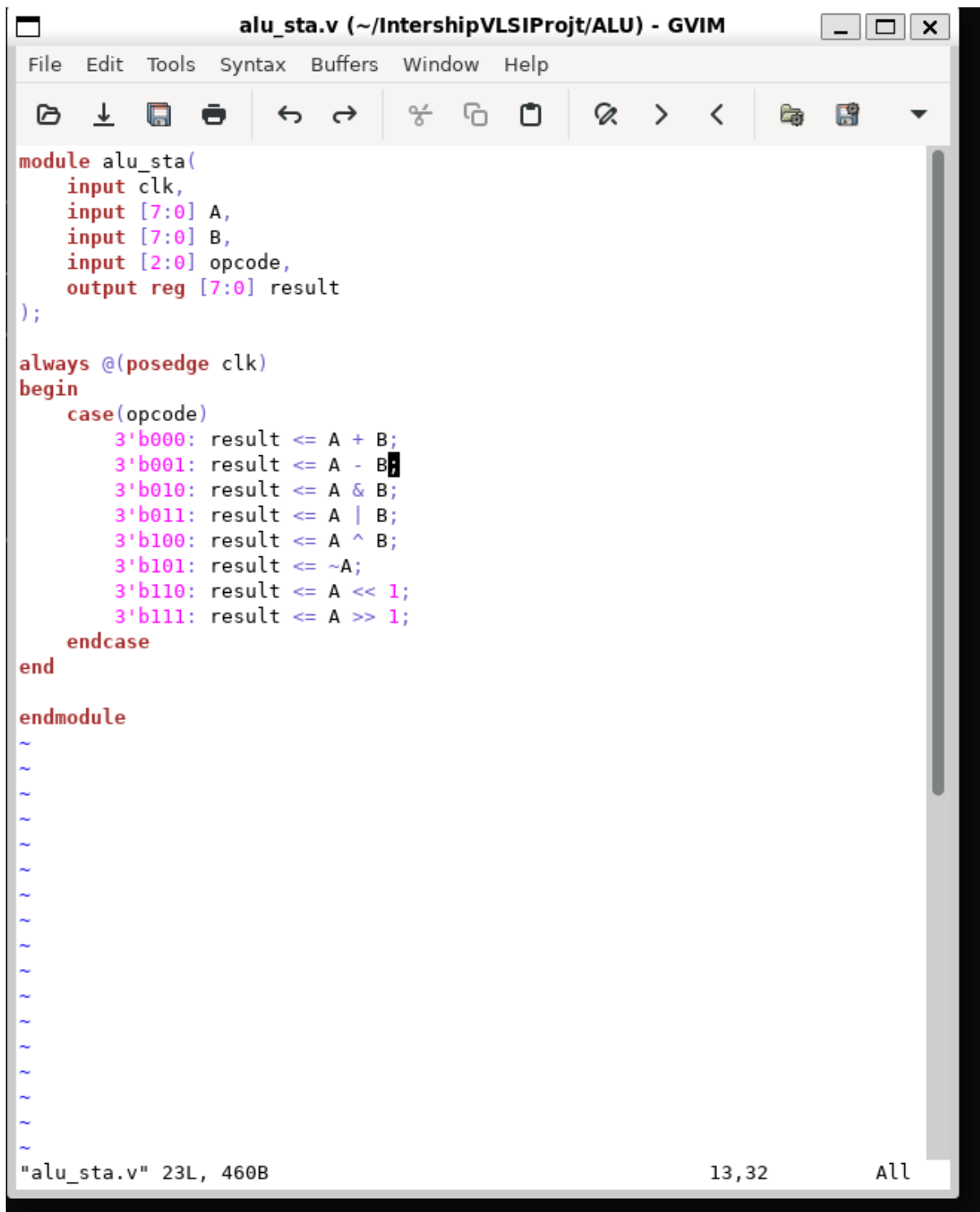
The synthesis flow completed successfully and a gate-level netlist was generated. OpenSTA was successfully installed and configured. During STA execution, a syntax compatibility issue was encountered while parsing the synthesized netlist generated by Yosys.

Observed Error:

"Syntax error while reading synthesized netlist"

Due to this parser compatibility issue, a complete setup and hold timing report could not be generated. However, the complete STA flow, including synthesis, netlist generation, timing constraints, and OpenSTA execution, was successfully demonstrated.

OUTPUT IMAGES :



```
govind@LAPTOP-HQSI9PA0:~/InterShipVLSIProjt/ALU$ nc -ba alu_sta_netlist_fix.v | sed -n '210,220p'
210     wire clk;
211     input [2:0] opcode;
212     wire [2:0] opcode;
213     output [7:0] result;
214     wire [7:0] result;
215     assign _130_ = B[0] | A[0];
216     assign _131_ = B[0] & A[0];
217     assign _132_ = _130_ & ~( _131_ );
218     assign _133_ = ~opcode[2];
219     assign _134_ = ~(opcode[1] & opcode[0]);
220     assign _135_ = _134_ | _133_;
govind@LAPTOP-HQSI9PA0:~/InterShipVLSIProjt/ALU$ gvim alu_sta.v
govind@LAPTOP-HQSI9PA0:~/InterShipVLSIProjt/ALU$ cat alu_sta.v
module alu_sta(
    input clk,
    input [7:0] A,
    input [7:0] B,
    input [2:0] opcode,
    output reg [7:0] result
);

always @(posedge clk)
begin
    case(opcode)
        3'b000: result <= A + B;
        3'b001: result <= A - B;
        3'b010: result <= A & B;
        3'b011: result <= A | B;
        3'b100: result <= A ^ B;
        3'b101: result <= ~A;
        3'b110: result <= A << 1;
        3'b111: result <= A >> 1;
    endcase
end

endmodule
govind@LAPTOP-HQSI9PA0:~/InterShipVLSIProjt/ALU$ gvim synth_sta.js
govind@LAPTOP-HQSI9PA0:~/InterShipVLSIProjt/ALU$
```

```
26. Executing CHECK pass (checking for obvious problems).
Checking module alu_sta...
Found and reported 0 problems.

Executing Verilog backend.

1. Executing BMUXMAP pass.

2. Executing DEMUXMAP pass.
Mapping module '\alu_sta'.

d of script. Logfile hash: e1cc0a7a54, CPU: user 0.16s system 0.02s, MEM: 13.7
sys 0.33 (git sha1 2584903a060)
me spent: 33% 1x abc (0 sec), 14% 17x opt_expr (0 sec), ...
vind@LAPTOP-HQSI9PA0:~/InterShipVLSIProjt/ALU$ head -20 alu_sta_netlist.v
Generated by Yosys 0.33 (git sha1 2584903a060) */

module alu_sta(clk, A, B, opcode, result);
wire _000_;
wire _001_;
wire _002_;
wire _003_;
wire _004_;
wire _005_;
wire _006_;
wire _007_;
wire _008_;
wire _009_;
wire _010_;
wire _011_;
wire _012_;
wire _013_;
wire _014_;
wire _015_;
wire _016_;
vind@LAPTOP-HQSI9PA0:~/InterShipVLSIProjt/ALU$ yosys -V
sys 0.33 (git sha1 2584903a060)
vind@LAPTOP-HQSI9PA0:~/InterShipVLSIProjt/ALU$ gvim sta_sta.tcl
vind@LAPTOP-HQSI9PA0:~/InterShipVLSIProjt/ALU$
```

```
Time spent: 33% 1x abc (0 sec), 14% 17x opt_expr (0 sec), ...
govind@LAPTOP-HQSI9PA0:~/IntershipVLSIProjt/ALU$ head -20 alu_sta_netlist.v
/* Generated by Yosys 0.33 (git sha1 2584903a060) */
```

```
module alu_sta(clk, A, B, opcode, result);
  wire _000_;
  wire _001_;
  wire _002_;
  wire _003_;
  wire _004_;
  wire _005_;
  wire _006_;
  wire _007_;
  wire _008_;
  wire _009_;
  wire _010_;
  wire _011_;
  wire _012_;
  wire _013_;
  wire _014_;
  wire _015_;
  wire _016_;
govind@LAPTOP-HQSI9PA0:~/IntershipVLSIProjt/ALU$ yosys -V
Yosys 0.33 (git sha1 2584903a060)
govind@LAPTOP-HQSI9PA0:~/IntershipVLSIProjt/ALU$ gvim sta_sta.tcl
govind@LAPTOP-HQSI9PA0:~/IntershipVLSIProjt/ALU$ cat constraints_sta.sdc
create_clock -name clk -period 10 [get_ports clk]
govind@LAPTOP-HQSI9PA0:~/IntershipVLSIProjt/ALU$ |
```

```
govind@LAPTOP-HQSI9PA0:~/IntershipVLSIProjt/ALU$ nl -ba alu_sta_netlist_fix.v | sed -n '210,220p'
210   wire clk;
211   input [2:0] opcode;
212   wire [2:0] opcode;
213   output [7:0] result;
214   wire [7:0] result;
215   assign _130_ = B[0] | A[0];
216   assign _131_ = B[0] & A[0];
217   assign _132_ = _130_ & ~(_131_);
218   assign _133_ = ~opcode[2];
219   assign _134_ = ~(opcode[1] & opcode[0]);
220   assign _135_ = _134_ | _133_;
govind@LAPTOP-HQSI9PA0:~/IntershipVLSIProjt/ALU$ gvim alu_sta.v
govind@LAPTOP-HQSI9PA0:~/IntershipVLSIProjt/ALU$ cat alu_sta.v
module alu_sta(
  input clk,
  input [7:0] A,
  input [7:0] B,
  input [2:0] opcode,
  output reg [7:0] result
);

always @(posedge clk)
begin
  case(opcode)
    3'b000: result <= A + B;
    3'b001: result <= A - B;
    3'b010: result <= A & B;
    3'b011: result <= A | B;
    3'b100: result <= A ^ B;
    3'b101: result <= ~A;
    3'b110: result <= A << 1;
    3'b111: result <= A >> 1;
  endcase
end

endmodule
govind@LAPTOP-HQSI9PA0:~/IntershipVLSIProjt/ALU$ gvim synth_sta.py
govind@LAPTOP-HQSI9PA0:~/IntershipVLSIProjt/ALU$ yosys -s synth_sta.py
```